📄 جزوه سیستم عامل - فصل ۷: مثال‌های همگامی

(بخش سوم: همگامی در پوزیکس POSIX)

۱. مقدمه: چرا POSIX؟

ابزارهایی که در بخش قبل دیدیم (مثل قفل چرخشی در ویندوز یا atomic_t در لینوکس) ابزارهای سطح هسته بودند و برنامه‌نویسان معمولی به آن‌ها دسترسی ندارند.

- **رابط POSIX:** یک استاندارد جهانی (برای سیستم‌های یونیکس، لینوکس و macOS) است که ابزارهای همگامی را در **سطح کاربر (User Level)** فراهم می‌کند.
- کتابخانه اصلی: pthread.h (برای نخ‌ها) و semaphore.h (برای سمافورها).

۲. قفل‌های انحصار متقابل (POSIX Mutex Locks)

ساده‌ترین و پایه‌ای‌ترین ابزار همگامی در پوزیکس.

نوع داده و توابع اصلی:

- **نوع داده:** pthread_mutex_t
- **ایجاد و مقداردهی:** pthread_mutex_init(&mutex, NULL);
- **گرفتن قفل (Acquire):** pthread_mutex_lock(&mutex);
  - اگر قفل آزاد نباشد، نخ مسدود (Block) می‌شود.
- **آزاد کردن قفل (Release):** pthread_mutex_unlock(&mutex);

الگوریتم حفاظت از بخش بحرانی:

```
<include <pthread.h#

;pthread_mutex_t mutex
(&mutex, NULL); // ساخت قفل ;pthread_mutex_init

/* ... در کد نخ‌ها ... */

(&mutex); // ۱ . ورود ممنوع! (قفل کن) pthread_mutex_lock

/* --- بخش بحرانی (Critical Section) --- */

(&mutex); // ۲ . بفرما! (آزاد کن) pthread_mutex_unlock
```

۳. سمافورهای پوزیکس (POSIX Semaphores)

سمافورها در استاندارد اصلی Pthreads نیستند ولی در افزونه POSIX SEM وجود دارند. دو نوع سمافور داریم:

الف) سمافورهای نام‌دار (Named Semaphores)

- **ویژگی:** یک "نام" (رشته متنی) دارند.
- **کاربرد:** می‌توانند بین **چند پردازه (Process) غیرمرتبط** به اشتراک گذاشته شوند (چون هر کس نامش را بداند، پیدایش می‌کند).
- **ایجاد:** ;(1 ,0666 ,sem_open("SEM_NAME", O_CREAT
  - پارامتر آخر (۱) مقدار اولیه سمافور است.

ب) سمافورهای بی‌نام (Unnamed Semaphores)

- **ویژگی:** نام ندارند و در حافظه قرار می‌گیرند.
- **کاربرد:**
  - بین **نخ‌های یک پردازه** (Thread Sharing).
  - یا در حافظه مشترک (Shared Memory) بین پردازه‌ها.
- **ایجاد:** ;(1 ,0 ,sem_init(&sem

- پارامتر دوم (۰): یعنی فقط بین نخ‌های همین پردازه مشترک باشد. (اگر غیر صفر باشد، بین پردازه‌ها).
- پارامتر سوم (۱): مقدار اولیه.

عملیات سمافور (برای هر دو نوع):

1. **Wait (انتظار/کاهش):** sem_wait(&sem);
2. **Signal (ارسال سیگنال/افزایش):** sem_post(&sem);

۴. متغیرهای شرط (POSIX Condition Variables)

زبان C مانیتور (Monitor) ندارد، اما می‌توانیم با ترکیب **Mutex** و **Condition Variable** رفتار مانیتور را شبیه‌سازی کنیم.

نوع داده و توابع:

- **نوع داده:** pthread_cond_t
- **ایجاد:** pthread_cond_init(&cond_var, NULL);

عملیات اصلی:

انتظار (Wait): pthread_cond_wait(&cond_var, &mutex);

- **نکته بسیار مهم:** این تابع باید حتماً زمانی صدا زده شود که mutex قفل است.
- **کاری که انجام می‌دهد:** به صورت اتمیک، قفل mutex را آزاد می‌کند و نخ را به خواب می‌برد. وقتی بیدار شد، دوباره mutex را قفل می‌کند.

سیگنال (Signal): pthread_cond_signal(&cond_var);

- یکی از نخ‌های منتظر را بیدار می‌کند.

الگوریتم استاندارد استفاده (بسیار مهم برای کدنویسی):

فرض کنید می‌خواهیم منتظر بمانیم تا شرط a == b برقرار شود.

کد نخ منتظر (Waiting Thread):

کد نخ منتظر (Waiting Thread):

```
// ۱ pthread_mutex_lock(&mutex); حتما اول قفل کن
// ۲ while (a != b) { تا وقتی شرط برقرار نیست
// ۳ pthread_cond_wait(&cv, &mutex); قفل را بده و بخواب (منتظر شو)
// وقتی بیدار شدی، دوباره شرط را چک کن (چون شاید سیگنال کاذب باشد)
{
/* کار با داده‌ها */
// ۴ pthread_mutex_unlock(&mutex); قفل را آزاد کن
```

کد نخ سیگنال‌دهنده (Signaling Thread):

```
// ۱ pthread_mutex_lock(&mutex); قفل کن
// ۲ a = b; داده را تغییر بده (شرط را برقرار کن)
// ۳ pthread_cond_signal(&cv); خبر بده!
// ۴ pthread_mutex_unlock(&mutex); آزاد کن
```

۵. خلاصه نکات کلیدی فصل ۷ (جمع‌بندی نهایی)

مسائل کلاسیک:

- **بافر محدود:** مشکل تولیدکننده-مصرف‌کننده. حل با ۳ سمافور (mutex, full, empty).
- **خوانندگان-نویسندگان:** نویسنده انحصار کامل می‌خواهد، خوانندگان اشتراکی. حل با سمافور/قفل rw_mutex.
- **فلاسفه:** مشکل بن‌بست سر میز غذا. حل با مانیتور (چک کردن دو همسایه قبل از خوردن).

همگامی ویندوز:

- در تک‌پردازنده: غیرفعال‌سازی وقفه.
- در چندپردازنده: Spinlock (که Preempt نمی‌شود).
- ابزار کاربر: Dispatcher Objects (میوتکس، سمافور، رویداد).

همگامی لینوکس:

- ساده‌ترین ابزار: atomic_t (اعداد اتمیک).
- Spinlock در تک‌پردازنده فقط Preemption را غیرفعال می‌کند.
- متغیر preempt_count نشان می‌دهد چند قفل داریم (اگر > 0 باشد، هسته متوقف نمی‌شود).

همگامی POSIX:

- استاندارد برنامه‌نویسی سطح کاربر.
- **Mutex:** برای انحصار متقابل ساده.
- **Semaphore:** نام‌دار (بین پردازه‌ها) و بی‌نام (بین نخ‌ها). توابع sem_wait, sem_post.
- **Condition Variable:** برای انتظار روی شرط خاص. همیشه همراه با یک Mutex استفاده می‌شود. تابع pthread_cond_wait.

پایان فصل هفتم